

SoDam-Reverse-Eng (소담-리버스) — 내 코드를 쉬운 한국어로 설명해 주는 도구

컴퓨터·AI가 처음이어도 괜찮습니다. 이 문서를 처음부터 따라가면 됩니다.

내가 만든(또는 허가받은) 코드·앱이 어떻게 동작하는지 **AI가 쉬운 한국어로 설명해** 주는 Claude Code 플러그인입니다.

⚠️ 정직한 약속: 크랙·불법 복제·비밀번호 추출·보안 우회 안내는 **요청해도 거부**합니다. 방어·교육·본인 소유물 분석 **전용**입니다.

목차

1. 이게 뭔가요?
2. 무엇을 해 주나요?
3. 무엇을 거부하나요?
4. SoDam 6형제 시너지
5. 사전 준비물
6. 설치 방법
7. 명령어
8. 분석 흐름 (워크플로우)
9. 보안·데이터 흐름
10. 아키텍처·파일 구조
11. 파일·문서 위치
12. 문제 해결·FAQ
13. 라이선스·저작권·상업적 용도
14. 연락·기여

1. 이게 뭔가요?

플러그인이란 기존 프로그램에 새로운 기능을 추가하는 작은 도구입니다. (스마트폰에 앱을 추가 설치하는 것과 비슷한 개념입니다.)

SoDam-Reverse-Eng는 Claude Code라는 AI 도구에 “코드 분석” 기능을 추가하는 플러그인입니다.

간단히 말하면

- AI에게 “이 코드가 뭐 하는 거야?” 라고 물으면
- AI가 코드를 읽고 **쉬운 한국어로 설명**해 줍니다
- 분석 결과는 **표준 보고서**로 깔끔하게 정리됩니다
- 위험한 요청(크랙·우회 등)은 **자동으로 거부**합니다

어디에 써요?

- 내가 예전에 만든 코드가 어떻게 동작하는지 다시 파악하고 싶을 때
- 다른 사람이 만든 코드(허가받은 것)를 이해하고 싶을 때
- “이 함수가 왜 이렇게 작동하지?” 궁금할 때
- 내 코드의 보안 취약점이 어디 있는지 찾고 싶을 때

2. 무엇을 해 주나요?

기능	설명
 코드 설명	“이 함수가 뭐 하는 거야?” → 한국어로 풀어서 설명
 표준 보고서 자동 출력	요약 · 함수별 설명 · 근거 위치 · 불확실한 점 · 다음 확인사항
 안전 3층 보호	AI 거부(1층) + 위험 차단(2층) + 파일 변조 감지(3층) 동시 작동
 자동 마스킹	비밀번호 · API키 · 토큰이 보고서에 평문 노출되지 않도록 처리
 전체 한국어 안내	모든 안내 · 오류 메시지가 한국어
 로컬 전용 저장	분석 결과를 내 컴퓨터에만 저장 (외부 서버로 전송 없음)

3. 무엇을 거부하나요? (안전 경계)

요청	결과	이유
“이 코드가 어떻게 동작해?”	✅ 분석 진행 (동의 후)	허용 범위
“버그 찾아줘”	✅ 분석 진행 (동의 후)	허용 범위
“보안 취약점 어디야?”	✅ 분석 진행 (동의 후)	허용 범위
“크랙 방법 알려줘”	❌ 거부	절대 금지
“인증 우회 코드 만들어줘”	❌ 거부	절대 금지
“토큰/비밀번호 추출해줘”	❌ 거부	절대 금지
“DRM 해제 방법은?”	❌ 거부	절대 금지
“라이선스 우회해줘”	❌ 거부	절대 금지
분석 중 발견한 API 키	🔒(마스킹 됨) 표시	개인정보 보호

4. SoDam 6형제 시너지

SoDam-Reverse는 6형제 플러그인 중 **막내**입니다. 형제들과 함께 설치하면 더 강력해 집니다.

권장 설치 순서: Harness → Loop → Context → Agentic → Prompt → Reverse(이것)

형제 플러그인	역할	Reverse와 함께 쓰면
SoDam-Harness	안전벨트 · 백업	RE 위험 패턴 공유 → hook 1개가 모두 담당 (중복 없음)
SoDam-Loop	반복 작업 안전 제어	반복 분석 루프도 안전하게
SoDam-Context	CLAUDE.md 건강검진	분석 범위 이탈 자동 감지
SoDam-Agentic	계획 · 쉬운 검토	보고서를 비개발자 눈높이로 재검수
SoDam-Prompt	자연어 요청 개선	“이거 분석해줘” 품질 향상

⚠️ **중요:** C:\Users\이름 (홈 폴더)에서 Claude Code를 시작하면 Harness · Loop guard가 정상 작업까지 막습니다. 반드시 **프로젝트 폴더** (D:\내프로젝트\등)에서 시작하세요.

Harness가 설치된 경우 — 한 번만 실행:

```
node scripts/re-inject-harness.mjs
```

→ RE 위험 패턴이 Harness 안전규칙에 추가되어 hook을 공유합니다 (중복 없음).

5. 사전 준비물 (무료, 2가지)

준비물 1: Node.js (버전 18 이상)

Node.js란 이 플러그인의 안전장치가 동작하는 데 필요한 “실행 엔진”입니다.

이미 있는지 확인: 1. 키보드에서 **Windows 키** 누르기 2. powershell 입력 → **Windows PowerShell** 클릭 3. 검은 창이 열리면 아래를 입력하고 **Enter**: `node --version` 4. v18. 이상 숫자가 나오면 **OK** (예: v20.19.0, v22.14.0)

없다면 (또는 v18보다 낮다면) 설치 방법: 1. 웹브라우저(크롬, 엣지 등) → **nodejs.org** 접속 2. 초록색 “**LTS**” 버튼 클릭 → 설치 파일(.msi) 내려받기 3. 내려받은 파일 더블클릭 → “**다음(Next)**”을 계속 누르기 → 설치 완료 4. PowerShell 창 **완전히 닫기** → 새로 열기 5. 다시 `node --version` 입력하여 버전 확인

💡 컴퓨터를 재부팅해야 반영되는 경우도 있습니다.

준비물 2: Claude Code

이 문서를 Claude Code에서 보고 있다면 **이미 있는 것**입니다. 없다면 Anthropic 공식 사이트에서 Claude Code를 설치하세요.

6. 설치 방법

📌 더 자세한 단계별 안내: [GUIDE.md](#) 2장 참고

⚠️ 설치 전 반드시 읽기 — 가장 흔한 실패 원인

명령어가 안 뜨는 원인 1위: Claude Code를 잘못된 폴더에서 시작했을 때

Claude Code는 **컬 때의 폴더**를 기준으로 명령어를 읽습니다. 채팅창에서 `cd` 다른 폴더를 입력해도 명령어는 다시 읽히지 않습니다.

올바른 순서: 1. PowerShell(검은 창)을 열고 2. `cd D:\내프로젝트폴더` 입력 (프로젝트 폴더로 이동) 3. `claude` 입력해서 Claude Code 실행

❌ 홈 폴더(C:\Users\이름)에서 실행하면 /re-start 명령이 안 됩니다.

단계 1: 마켓플레이스 등록

Claude Code 채팅창에 아래를 입력하세요:

📁 이 플러그인은 비공개 저장소입니다. 아래 두 방법 중 받은 방식에 맞는 것을 선택하세요.

방법 A — GitHub 저장소에서 복제 (저장소 초대를 받은 분)

⚠️ GitHub 계정이 필요하며, 저장소 접근 권한(초대)이 있어야 합니다. 초대 요청: startmxk@gmail.com

1. PowerShell(검은 창)을 열고 복제 명령어 입력:

```
git clone https://github.com/sodam-ai/SoDam-Reverse-Eng.git
```

→ 현재 폴더 안에 SoDam-Reverse-Eng 폴더가 생성됩니다.

2. 복제된 폴더 경로 확인 (예: C:\Users\내이름\SoDam-Reverse-Eng)
3. Claude Code 채팅창에 해당 경로로 입력:

```
/plugin marketplace add C:\Users\내이름\SoDam-Reverse-Eng
```

💡 내이름 부분을 실제 사용자 이름으로 바꿔 입력하세요.

방법 B — 폴더/압축 파일로 전달받은 분 (git 불필요)

1. 받은 zip 파일의 압축을 풀거나 폴더를 원하는 위치에 저장 (예: C:\Users\내이름\SoDam-Reverse-Eng)
2. Claude Code 채팅창에 실제 경로 입력:

```
/plugin marketplace add C:\Users\내이름\SoDam-Reverse-Eng
```

💡 경로에 공백이 포함된 경우 큰따옴표로 감싸세요: /plugin marketplace add "C:\내 이름 포함 경로\SoDam-Reverse-Eng"

단계 2: 플러그인 설치

마켓플레이스 등록 후 아래를 입력:

```
/plugin install sodam-reverse@sodamreverse-marketplace
```

또는 /plugin 입력 → 메뉴에서 **Browse marketplaces** → **sodam-reverse** → **Install** 선택.

단계 3: 완전 재시작 (반드시!)

플러그인은 Claude Code **시작 시에만 로드**됩니다. 반드시 완전 종료 후 재시작해야 합니다.

1. 채팅창에 /exit 입력 → Enter
2. 터미널(검은 창) **완전히 닫기**
3. PowerShell 새로 열기
4. **반드시 프로젝트 폴더에서 시작:**

```
cd D:\내 프로젝트 폴더
claude
```

단계 4: 설치 확인

Claude Code가 열리면 /re-p 입력 → 자동완성에 아래가 떴어야 정상:

```
/re-ping    (sodam-reverse) ← 진단용
/re-start   (sodam-reverse) ← 분석 시작
```

(sodam-reverse) 꼬리표가 붙어 있으면 플러그인으로 제대로 로딩된 것입니다.

진단 명령어 실행:

```
/re-ping
```

→ "Pong! /re-ping 정상 작동합니다." 응답이 오면 설치 완료.

단계 5: 안전 3층 검증 (필수!)

```
/re-selftest
```

기대 결과:    3개 모두 초록색

초록색 3개가 나오지 않으면 → [GUIDE.md](#) 2장 참고

단계 6: 무결성 해시 등록 (3층 활성화)

selftest 출력에서 SHA-256 해시를 복사 → references/integrity.json 파일의 hooks/re-deny-guard.mjs 값에 붙여넣기 → 저장 → 다시 /re-selftest 실행 → 3층  확인

7. 명령어

명령어	언제 사용하나요	예시
/re-ping	설치 확인 · 진단용 (가장 먼저 테스트)	/re-ping
/re-start [파일경로]	새로운 분석을 시작할 때	/re-start src/login.js
/re-report	마지막 분석 보고서를 다시 볼 때	/re-report
/re-selftest	안전장치 3층 점검할 때	/re-selftest

앞으로 추가될 명령어 (Phase 2·3 구현 후):

명령어	상태	설명
/re-android [APK경로]	 Phase 2 예정	Android 앱 분석
/re-binary [파일경로]	 Phase 3 예정	바이너리/실행파일 분석

8. 분석 흐름 (워크플로우)

사용자: /re-start 내코드/login.js

↓

[1단계] 동의 게이트

AI: "이 코드가 본인 소유이거나 허가받은 것입니까?" → 예/아니오

AI: "방어·교육 목적에 동의합니까? 책임은 본인에게 있습니다." → 예/아니오
둘 다 "예"여야 다음 단계로 진행

↓

[2단계] 안전 3층 점검 (자동)

- 1층: AI가 크랙·우회 내용인지 자체 판단
- 2층: deny-hook이 위험 패턴 실시간 차단
- 3층: 안전파일 변조 여부 SHA-256으로 확인

↓

[3단계] 분석 시작 (읽기 전용)

- 파일 읽기만 (절대 실행 안 함)
- 경로 조작 (../, 심볼릭 링크) 자동 차단
- 비밀번호 · 키 발견 시 자동 마스킹

↓
[4단계] 표준 보고서 출력

- 한 줄 요약 (이 코드가 하는 일 1줄로)
 - 함수별 설명 (파일명:줄번호 근거 포함)
 - 불확실한 점 (AI가 확신 못하는 부분 명시)
 - 다음 확인사항 (추가로 볼 것 제안)

↓
[5단계] 로컬 저장
.sodam-re/ 폴더에 저장 (.gitignore 등록됨)
외부 서버 전송 없음

9. 보안·데이터 흐름

내 코드 데이터는 어디로 가나요?

내 컴퓨터의 코드
↓ (분석을 위해 AI에 전송)
Claude AI (Anthropic 서버)
↓ (분석 결과만 반환)
내 컴퓨터 .sodam-re/ 저장

- **AI에 전송:** 코드 내용 (분석을 위해 필요 — Claude API 이용 약관 적용)
- **외부 전송 안 됨:** 분석 결과, 동의 기록, 차단 로그
- **마스킹 처리 후 저장:** 코드 중 발견된 비밀번호·키·토큰

3층 안전장치 구조

층	이름	역할	파일
1층	AI 거부 규칙	AI 자체가 크랙·우회 내용 출력 거부	skills/re-router/SKILL.md
2층	deny-hook	위험 키워드·패턴 실시간 차단	hooks/re-deny-guard.mjs
3층	무결성 점검	안전파일이 변조됐는지 SHA-256으로 확인	hooks/_selftest.mjs

fail-closed 원칙: hook에 오류가 생기면 “통과”가 아니라 **분석 즉시 중단**입니다.

분석 결과 저장 위치

[플러그인 폴더]/

└ .sodam-re/	← 분석 결과 저장 폴더 (자동 생성)
└ reports/	← 보고서 파일들
└ consent/	← 동의 기록 (con-TIMESTAMP.json)
└ safety-log.jsonl	← 차단 이벤트 기록 (원문 해시 처리됨)

.sodam-re/ 폴더는 .gitignore에 등록되어 **Git에 업로드되지 않습니다.**

10. 아키텍처·파일 구조

SoDam-Reverse-Eng/

└ .claude-plugin/	← Claude Code 플러그인 선언
└ plugin.json	
└ marketplace.json	
└ commands/	← 명령어 정의 (/re-start 등)
└ re-ping.md	← 진단 명령어
└ re-start.md	
└ re-report.md	
└ re-selftest.md	
└ re-android.md	← Phase 2 예정
└ re-binary.md	← Phase 3 예정
└ skills/	← 분석 AI 로직
└ re-router/	← 1층 안전규칙 + 요청 분류
└ re-analyze-mycode/	← 소스코드 분석
└ re-report/	← 보고서 생성
└ re-analyze-android/	← Phase 2 예정 (stub)
└ re-analyze-binary/	← Phase 3 예정 (stub)
└ hooks/	← 안전장치 (2층 · 3층)
└ re-deny-guard.mjs	← 2층: 위험 패턴 실시간 차단
└ _selftest.mjs	← 3층: SHA-256 무결성 점검
└ hooks.json	← hook 설정
└ references/	← 데이터·규칙 파일
└ deny-corpus.json	← 위험 키워드 43개 패턴
└ mask-patterns.json	← 마스킹 패턴 10개
└ trust-catalog.md	← 신뢰 도구 카탈로그 (15개 repo)
└ report-template.md	← 보고서 표준 양식
└ integrity.json	← 안전파일 SHA-256 해시 저장소
└ scripts/	← 유틸리티 스크립트
└ re-inject-harness.mjs	← Harness 시너지 설정
└ re-inject-context.mjs	← Context 시너지 설정
└ check-family.mjs	← 6형제 상태 확인
└ check-trust-freshness.mjs	← 도구 신뢰도 신선도 점검

mcp/	
└─ catalog.json	← Phase 2.3 외부 도구 설정
samples/	← 테스트용 예제 파일
└─ safe-login.js	← 정상 분석 테스트용
└─ deny-demo.txt	← 차단 테스트용
.sodam-re/	← 분석 결과 저장 (자동 생성, .gitignore)
README.md	← 이 파일 (한국어 개요)
README.en.md	← English overview
GUIDE.md	← 초보 완전 사용 설명서 (한국어)
GUIDE.en.md	← English detailed guide
TROUBLESHOOTING.md	← 오류 해결 가이드
CHECKPOINT.md	← 개발 진행 상태 (개발자용)
SETUP_BLOCKED_FILES.md	← 안전파일 코드 전문
LICENSE	← Apache-2.0 라이선스 전문
NOTICE	← 저작권 고지

3층 안전 흐름 요약:

- [1층] skills/ 규칙 → AI가 위험 출력 자체를 거부
[2층] hooks/re-deny-guard.mjs → 위험 도구 호출 즉시 차단
[3층] hooks/_selftest.mjs → SHA-256으로 파일 변조 감지

11. 파일·문서 위치 (전체 목록)

파일 / 문서	위치	목적
한국어 README	README.md	이 파일 (전체 개요)
영어 README	README.en.md	English overview
한국어 상세 가이드	GUIDE.md	초보 완전 사용 설명서
영어 상세 가이드	GUIDE.en.md	English detailed guide
오류 해결 가이드	TROUBLESHOOTING.md	실패 패턴 전체 해결법
안전파일 코드 전문	SETUP_BLOCKED_FILES.md	수동 설정 파일 코드
라이선스 원문	LICENSE	Apache-2.0 전문
저작권 고지	NOTICE	서드파티 고지
개발 진행 상태	CHECKPOINT.md	개발자용 체크포인트
위험 패턴 DB	references/deny-corpus.json	43개 차단 패턴
마스킹 패턴	references/mask-patterns.json	10개 마스킹 규칙

파일 / 문서	위치	목적
신뢰 도구 목록	references/trust-catalog.md	15개 도구 신뢰등급
보고서 표준 양식	references/report-template.md	보고서 형식 정의
무결성 해시	references/integrity.json	SHA-256 해시 저장
형제 상태 확인	scripts/check-family.mjs	6형제 진단 스크립트
신선도 점검	scripts/check-trust-freshness.mjs	신뢰 카탈로그 최신성 확인

12. 문제 해결·FAQ

Q1. /re-start 명령이 안 뜨거나 없어요 (가장 흔한 문제)

원인: Claude Code를 홈 폴더(c:\Users\이름)에서 실행했거나, 채팅창에서 cd로 이동한 것을 “이동했다”고 착각한 경우.

해결: 1. Claude Code 완전히 닫기 2. PowerShell 새로 열기 3. **프로젝트 폴더로 이동**
후 실행: powershell cd D:\내프로젝트폴더 claude 4. /re-ping 입력 → "Pong!" 응답 확인

Q2. /re-ping 이 뭔가요?

설치 확인용 진단 명령어입니다. "Pong! /re-ping 정상 작동합니다." 라고 응답이 오면 플러그인이 제대로 로딩된 것입니다. 이 명령이 안 되면 /re-start도 안 됩니다.

Q3. “Node.js를 찾을 수 없습니다” 오류

nodejs.org → **LTS** 설치 → 터미널 켜다 켜기 → node --version 확인

Q4. 정상 작업이 계속 막혀요

c:\Users\이름 (홈 폴더)에서 실행하고 있을 가능성이 큼니다.

cd D:\내프로젝트폴더
claude

프로젝트 폴더에서 Claude Code를 다시 시작하세요.

Q5. 동의 질문에서 막혀요

“예” 또는 “네” 또는 “동의합니다”처럼 **명확하게** 입력하세요. “그런 것 같아요”, “아마도요” 같은 애매한 답변은 동의로 처리되지 않습니다.

Q6. 크랙/우회 요청이 거부됐어요

정상 동작입니다. 이 도구는 방어·교육·본인 소유물 전용입니다.

Q7. /re-selftest 에서 일부 항목이 ✖

SETUP_BLOCKED_FILES.md에서 5개 파일이 모두 있는지 확인 → 없는 파일만 다시 만들기
→ Claude Code 완전 재시작 → 다시 /re-selftest

SHA-256 불일치:

```
node hooks/_selftest.mjs
```

출력 해시 → references/integrity.json 저장 → 재실행

Q8. 보고서에서 비밀번호가 그대로 보여요

버그입니다. references/mask-patterns.json에 해당 패턴이 없는 것입니다. 해당 보고서를 공유하지 마시고 GitHub Issues에 신고해 주세요.

Q9. 파일이 너무 커서 분석이 오래 걸려요

파일 하나씩 분석하세요:

```
/re-start src/auth.js
```

폴더 전체보다 파일 단위가 훨씬 잘 동작합니다.

Q10. Harness 시너지가 안 연결돼요

```
node scripts/re-inject-harness.mjs
```

“RE 규칙 N개 주입 완료” 메시지가 나오면 성공.

Q11. 더 자세한 오류 해결

→ [GUIDE.md](#) — FAQ 25가지 포함 상세 가이드 → [TROUBLESHOOTING.md](#) — 실패 패턴 전체 해결법

13. 라이선스·저작권·상업적 용도

적용 라이선스

Apache License 2.0 · © 2026 SoDam AI Studio

허용되는 것 (명시적으로 허용)

행위	가능 여부
개인적 사용	 허용
수정 · 변경	 허용 (변경 사실 명시 의무)
복제 · 포크	 허용
재배포	 허용 (라이선스·저작권 고지 포함 의무)
상업적 사용	 허용
특허 사용	 허용

반드시 해야 하는 것 (의무 사항)

의무	내용
라이선스 고지 보존	LICENSE 파일을 반드시 포함
저작권 고지 보존	© 2026 SoDam AI Studio 고지 유지
변경 사실 명시	원본에서 수정한 부분이 있으면 명시
NOTICE 파일 동봉	NOTICE 파일을 함께 배포

허용되지 않는 것 (주의)

행위	설명
“SoDam” 상표 무단 사용	허가 없이 상표로 사용 불가
보증 요구	“문제 없이 동작한다”는 보증 없음 (AS-IS 제공)
손해배상 청구	면책 조항 적용

타사 상표 고지

- “Claude”, “Claude Code”, “Anthropic” — Anthropic 소유 상표
- “IDA Pro” — Hex-Rays 소유 상표 (Phase 3 선택 기능)
- 위 상표들은 본 제품과 공식 제휴 또는 보증 관계 없음

상업적 사용 시 반드시 확인할 것

항목	확인 사항
이 도구 자체	Apache-2.0으로 상업적 사용 가능
분석 대상 소프트웨어	해당 소프트웨어의 EULA · 라이선스 별도 확인
Claude API 사용	Anthropic 이용 약관 별도 적용
IDA Pro (Phase 3, 선택)	상용 라이선스 별도 구매 필요

면책 조항

- “있는 그대로(AS-IS)” 제공
- 품질 · 성능 · 특정 목적 적합성에 대한 **보증 없음**
- 사용 결과에 대한 책임은 **사용자 본인**에게 있음
- “100% 안전”은 어떤 보안 도구도 보장할 수 없습니다

라이선스 전문: [LICENSE](#) · 저작권 고지: [NOTICE](#) 더 자세한 라이선스 해설: [GUIDE.md](#)
11장 참고

14. 연락 · 기여

- 버그 신고 · 기능 요청: [GitHub Issues](#) (저장소 초대된 분만 접근 가능) 또는 이메일
- 이메일: startmxk@gmail.com

- **GitHub 저장소:** <https://github.com/sodam-ai/SoDam-Reverse-Eng> (비공개 — 초대 필요)
 - **기여 방법:** 이메일로 먼저 연락 → 협의 후 진행 (비공개 저장소)
-

English version: [README.en.md](#) 상세 사용 설명서: [GUIDE.md](#) 오류 해결: [TROUBLESHOOTING.md](#)